

# H.264 / AVC

Fundamentos técnicos del estándar



Grupo de Tecnología Electrónica  
y Comunicaciones

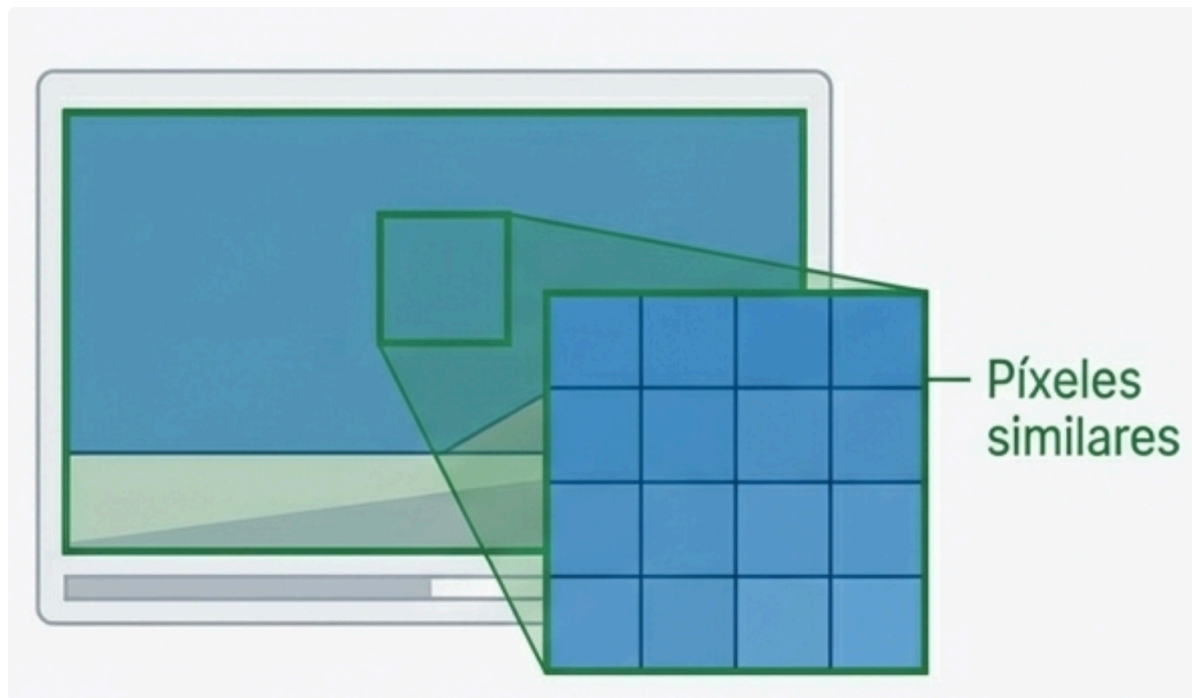


UNIVERSIDADE DA CORUÑA

UNIVERSIDADE DA CORUÑA

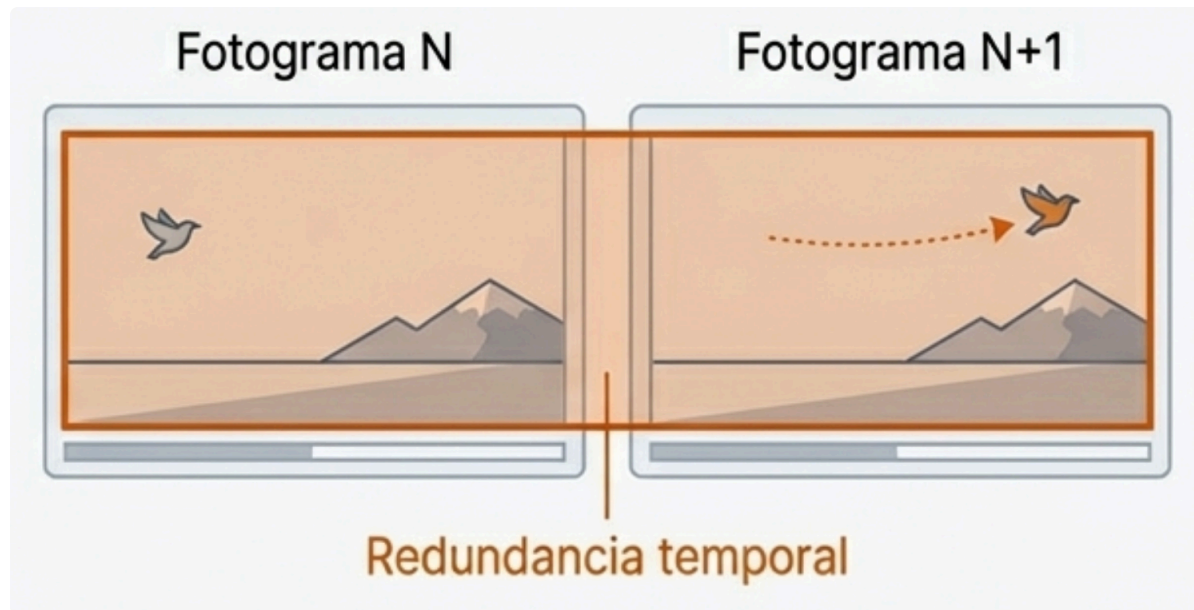
- Desarrollado por ITU-T y MPEG (grupo conjunto JVT).
- Publicado en 2003 como la evolución del vídeo digital de propósito general.
- Diseñado para ofrecer alta eficiencia tanto en bajo bitrate como en alta resolución.
- Actualmente sigue siendo uno de los códecs más usados:
  - Streaming (YouTube, Twitch, plataformas OTT)
  - Videoconferencia
  - DVB / IPTV
  - Almacenamiento y producción

- Correlación espacial en un frame.
  - Buscamos predecir los píxeles de un área a partir de los píxeles cercanos.



- Una predicción Intra será la predicción que se hace dentro del frame actual.
- Estándares antiguos como H.261 o H.263 no tenían una predicción Intra como tal.

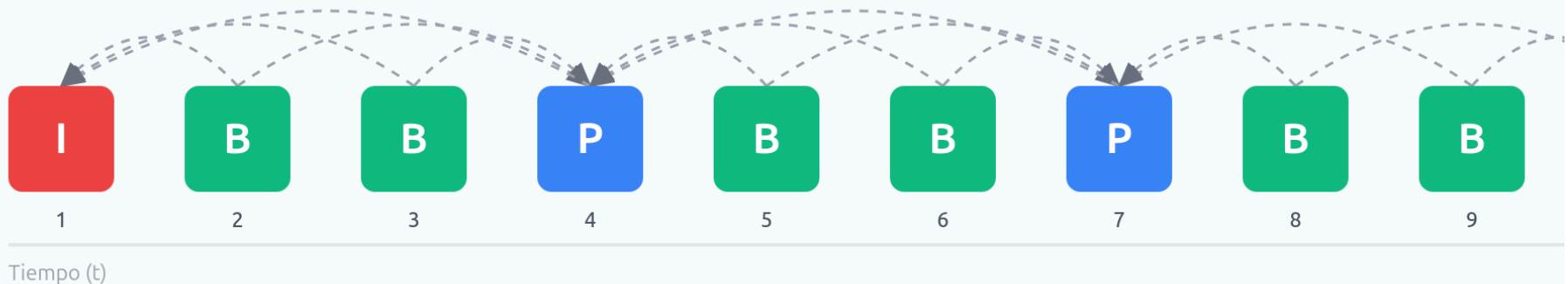
- Correlación temporal:
  - Buscamos predecir los píxeles de un área del frame actual a partir de los píxeles de otros frames (anteriores o posteriores).



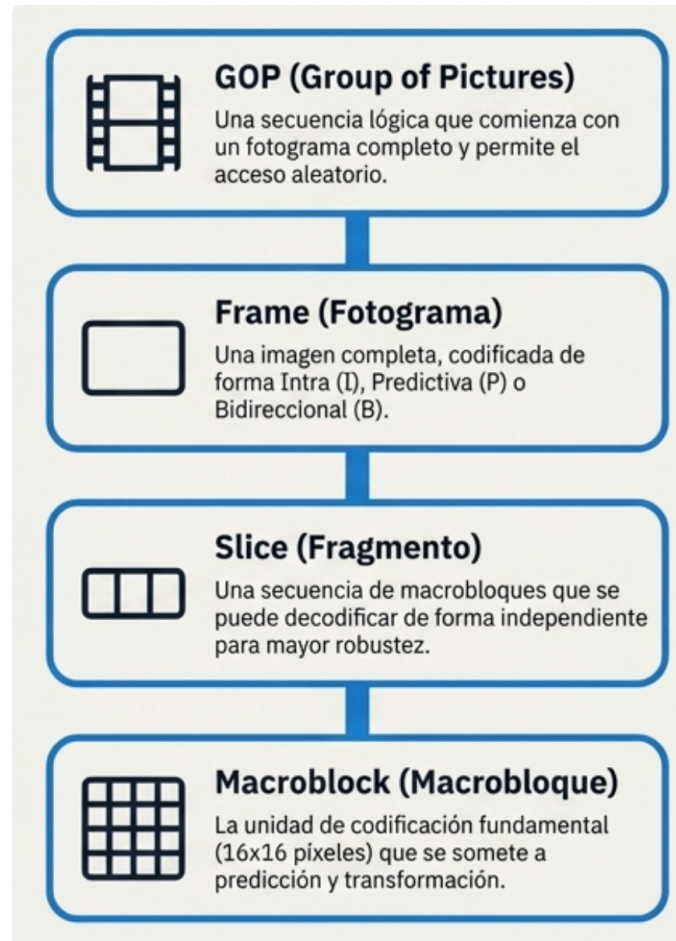
# Introducción

## Ideas principales

- Una predicción Inter será la predicción que se hace usando datos de otros frames.
- En estándares antiguos como H.261 el frame de referencia solo podía ser el anterior.
- En H.264 se pueden usar múltiples frames de referencia, tanto anteriores (lista 0, L0) como posteriores (lista 1, L1).



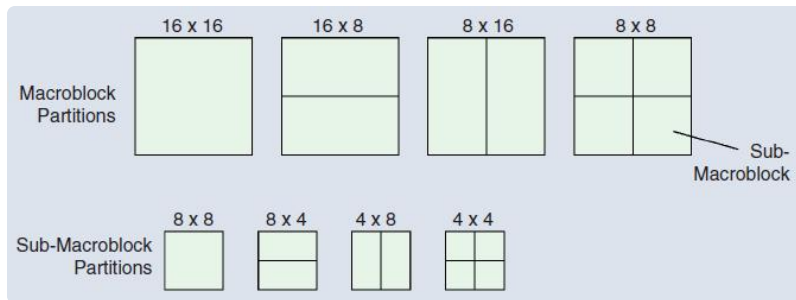
- Organización jerárquica:
  - Macroblock (MB) < Slice < Frame < GOP



- El MB es la unidad más pequeña de codificación.
- Un MB cubre:
  - 16×16 píxeles luma
  - 8×8 Cb + 8×8 Cr (submuestreo 4:2:0)
- Cada MB puede codificarse como:
  - Intra (dentro del frame)
  - Inter (predicción temporal usando datos de otros frames)
  - Skip (casi tal cual en otro frame)

## Estructura Macroblock (MB)

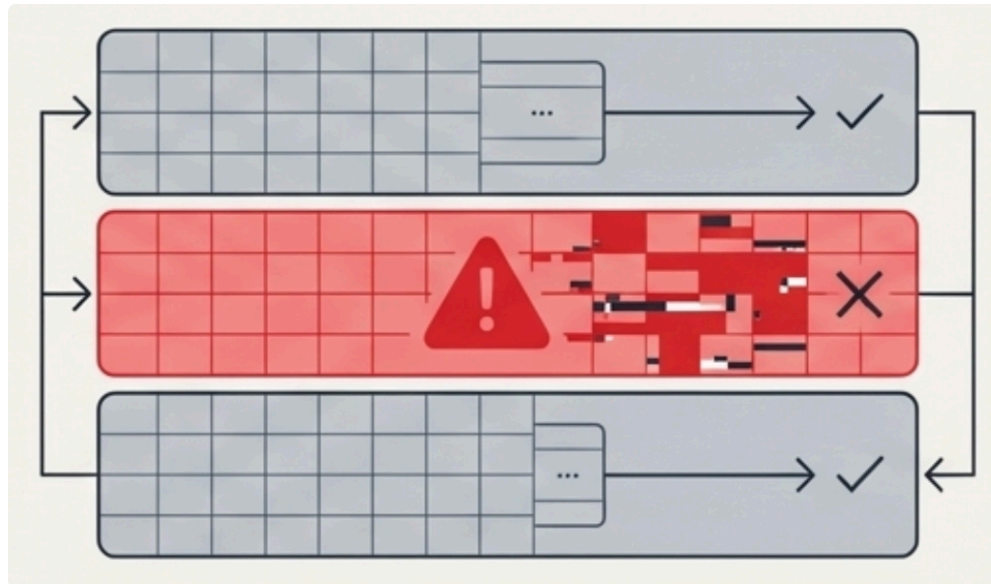
- H.264 permite dividir el MB para ajustar la predicción al contenido.
- En lugar de predecir con el MB de 16x16, se puede predecir con MBs de 16x8, 8x16, 8x8, 4x4, 4x8, 8x4.



- Objetivo:
  - Bloques grandes en zonas uniformes
  - Bloques pequeños en bordes y objetos en movimiento.
  - Predicción más precisa en zonas con movimiento.
  - Codificación más eficiente en zonas estáticas.

- Un slice es una secuencia de MBs dentro de una imagen que se codifica y decodifica sin depender de otros slices del mismo frame.
  - El estándar no define cuántos MBs puede tener un slice.
    - Incluso puede haber diferentes tamaños de slice en el mismo frame.
  - Se puede empezar a decodificar un slice sin haber decodificado los otros.

- Se usan para:
  - Dar robustez: si un slice no se puede decodificar, no afecta al resto de la imagen.
  - Paralelización: se pueden decodificar slices en paralelo.

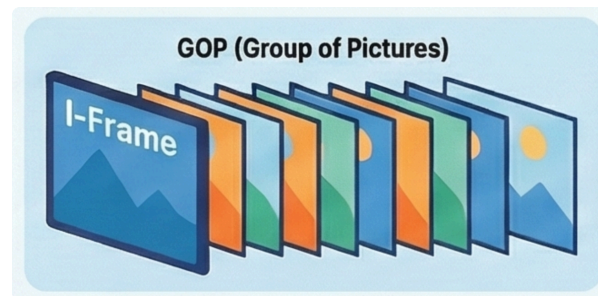


- En H.264 hay varios tipos de slice:
  - I-slice: solo MBs Intra.
    - normalmente en frames I.
  - P-slice: MBs con predicción Intra o Inter con referencia a frames anteriores.
  - B-slice: MBs con predicción Inter o Intra con referencia a frames anteriores y posteriores..
  - Hay también dos tipos slices especiales, SP y SI para conmutación de streams de diferentes calidades.
    - Pero hoy en día no se usan, hay tecnologías más avanzadas para ello.

- Un frame estará compuesto por uno o más slices.
  - Normalmente solo se usa uno por eficiencia.
- Según el tipo de codificación en los MBs que lo componen, los frames pueden ser:
  - I-frames
    - Codificados únicamente con predicción intra.
  - P-frames
    - Predicción temporal hacia atrás usando frames en la lista L0.
  - B-frames
    - Predicción bidireccional usando frames en la lista L0 (anteriores) y L1 (posteriores).

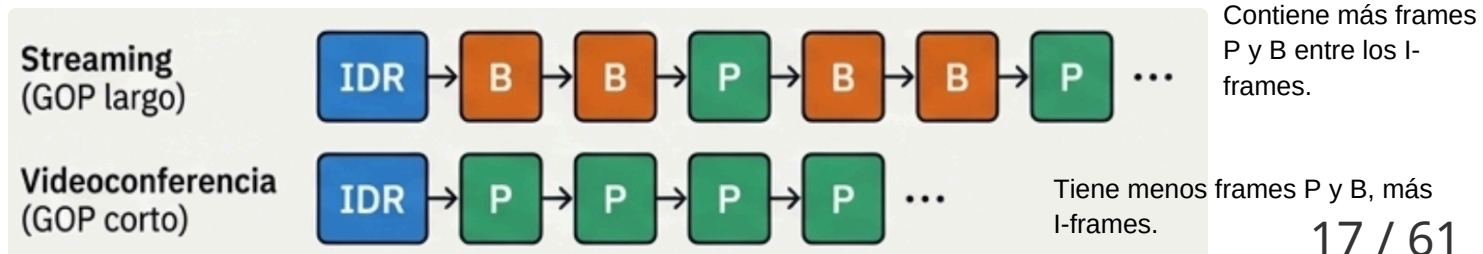
- Los I-Frame se usan para:
  - Inicio de secuencia.
  - Puntos de acceso aleatorio.
  - Frames con contenido complejo o con cambios abruptos.
  - Son los más costosos en términos de codificación.
- Los P-Frame:
  - Menos costosos que los I-Frames.
  - Utilizan predicción de movimiento.
  - Equilibrio entre calidad y tamaño.
- Los B Frames:
  - La mayor eficiencia.
  - Mayor latencia.

- Un GOP (Group of Pictures) es una unidad lógica consistente en un I-frame seguido de una secuencia de P-frames y B-frames.
- Sobre todo se usa para definir los puntos de acceso aleatorio, pero también permite controlar la eficiencia, latencia o robustez de la codificación.
- El estándar NO define una estructura GOP obligatoria.
  - Lo define el codificador, basándose en criterios como la latencia, tasa o tipo de contenido (streaming, videoconferencia, etc.).



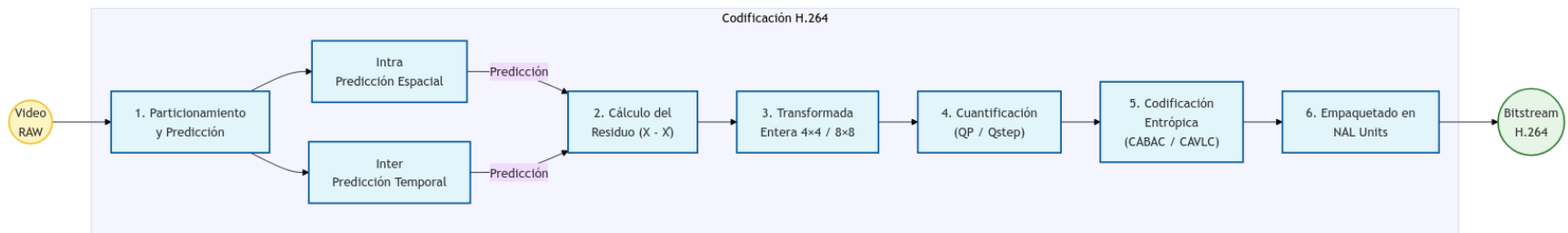
- Un GOP siempre empieza con un I-Frame, pero este puede ser en ocasiones un IDR (Instantaneous Decoding Refresh).
- Un IDR es un I-Frame, pero además:
  - Vacía completamente las listas de referencia, con lo que el decoder descarta TODOS los frames anteriores.
  - Ningún frame posterior puede referenciar a ningún frame previo al IDR.
  - Es un punto de acceso aleatorio garantizado.
  - Decodificar desde un IDR siempre da resultados correctos.

- Cómo el tamaño del GOP dice la distancia entre I-Frames, su valor es muy importante según el tipo de contenido:
  - GOP largo: 100–250 frames (streaming, broadcast)
  - GOP medio: 12–30 frames (producción, Blu-ray)
  - GOP muy corto: 1–5 frames (videoconferencia)
- El patrón concreto de tipos de frames es también a elección del codificador.
  - Pero no es fija, puede ser generada en base al contenido del video.



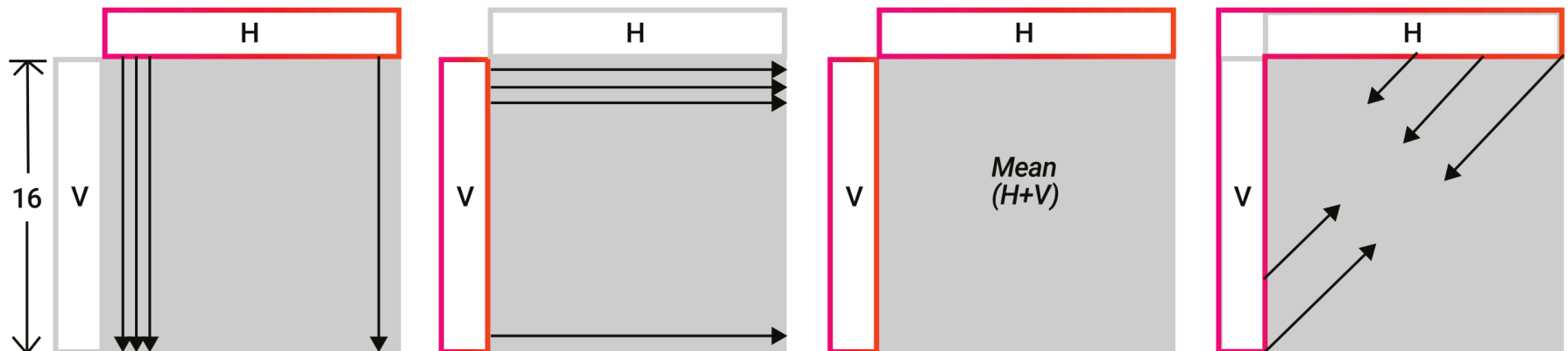
- El estandar define dos modos de GOP:
  - GOP cerrado:
    - Ningún frame fuera del GOP se usa como referencia.
    - Permite edición y cortes limpios.
  - GOP abierto:
    - Un B-frame al principio del GOP puede usar como referencia frames del GOP anterior.
    - Mayor eficiencia de compresión.
    - No permite cortes limpios entre GOPs.

- Para codificar con H.264, se pasa por diferentes etapas:

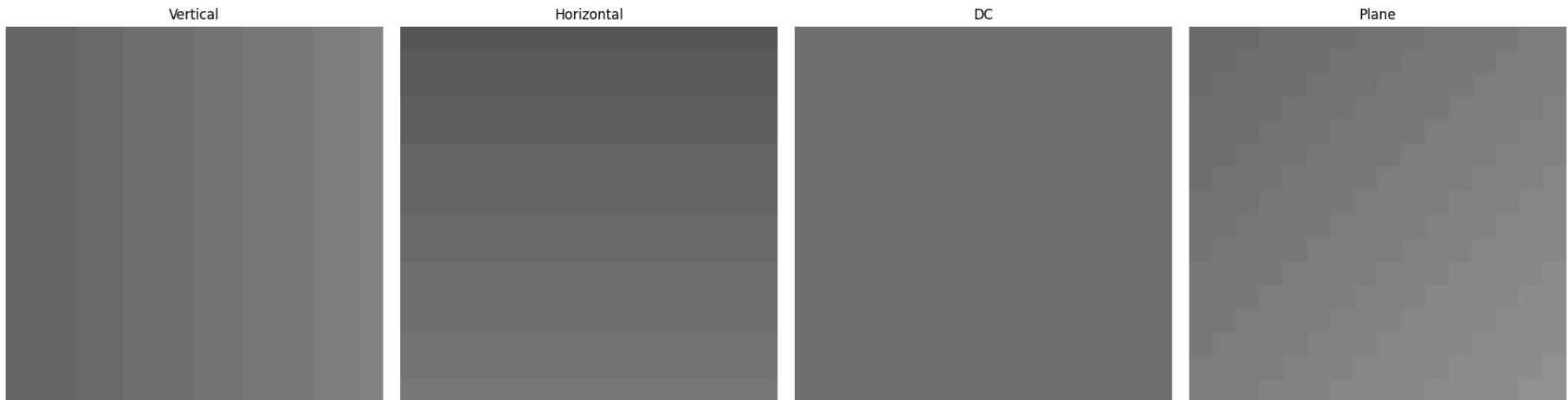


- A continuación iremos viendo cada una de ellas.

- La predicción Intra se hace dentro del mismo frame.
- Se predice un bloque a partir de los bloques vecinos reconstruidos.  
Si todos sus bloques vecinos son grises -> este también será gris
- Existen diferentes modos de predicción, unos para la parte de luminancia y otros para la parte de crominancia.
- Para áreas uniformes, en luminancia existen 4 modos que predicen directamente el valor del MB de 16x16.



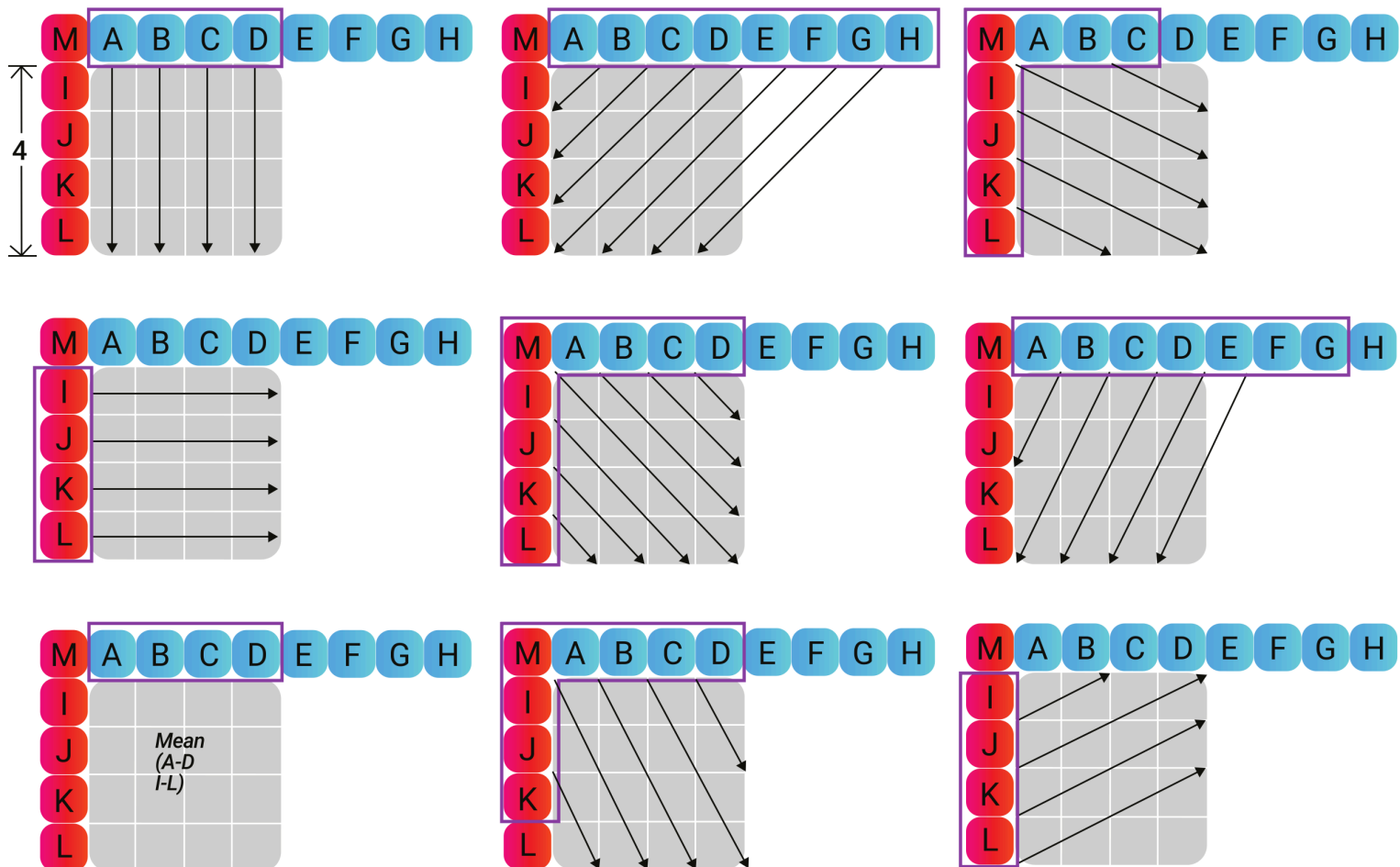
- A continuación se muestra un ejemplo aplicando los 4 modos usando los mismos píxeles de referencia a la izquierda y arriba.



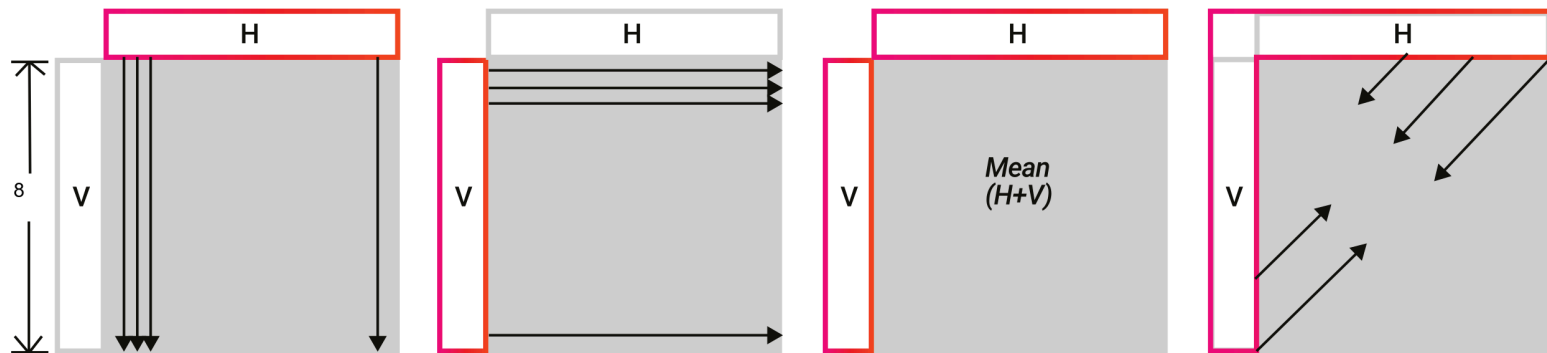
# Codificación

## Predicción Intra

- En áreas que tienen más detalle de luminancia, se usan más modos de predicción en bloques de 4x4:



- Para el color, existen 4 modos de predicción que son los mismos que para los bloques de 16x16 de luminancia, pero en este caso se usan bloques de 8x8.

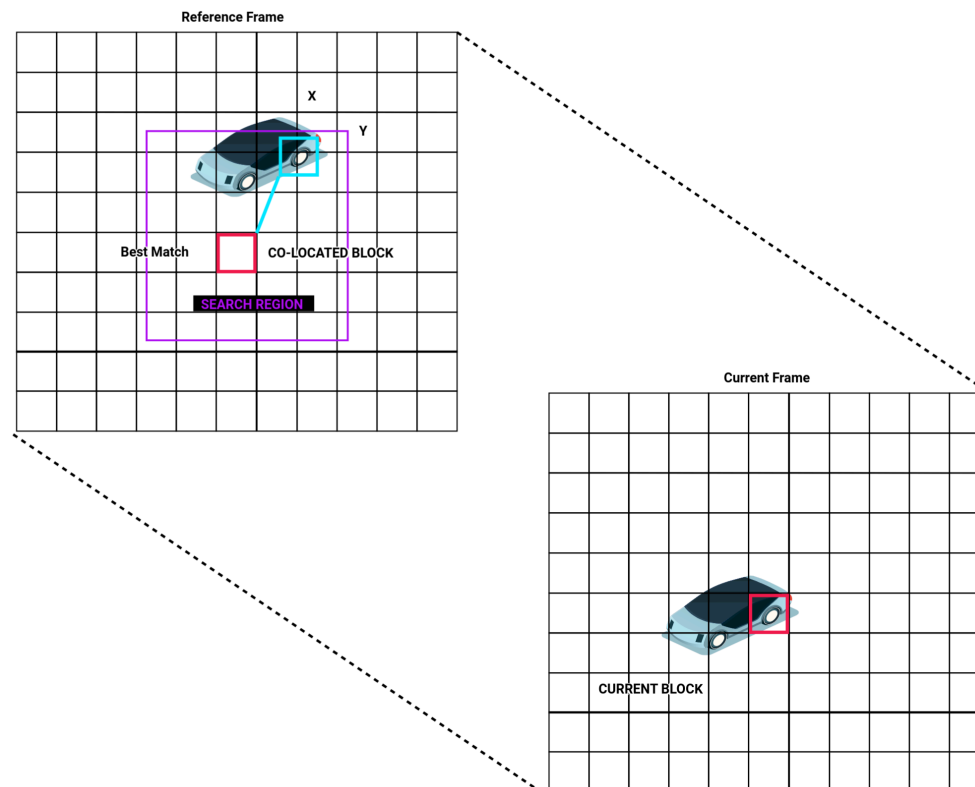


- Una vez que se ha hecho la predicción, se obtiene un residuo:
  - $\text{Residuo} = \text{Bloque original} - \text{Bloque predicho}$
- Lo que se codifica es el residuo, que debería tener valores cercanos a 0.
- Pero decodificador necesita también el modo de predicción usado.
  - Pero no se envía directamente.
  - Se usa el Most Probable Mode (MPM), que fue el modo usado en los bloques vecinos (arriba e izquierda).
  - Si el MPM coincide con el modo usado, se envía un único bit a 1 y ya está. Si no coincide, ese flag se pone a 0 y se envía el índice del modo usado.

# Codificación

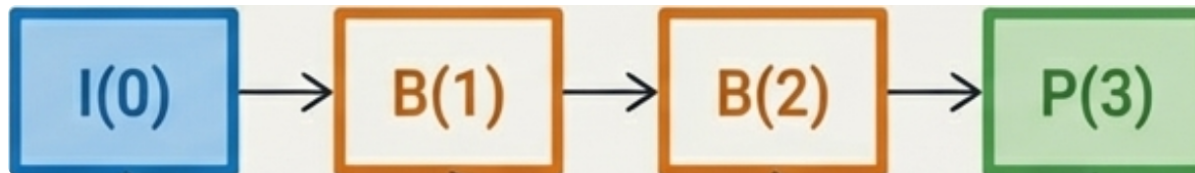
## Predicción Inter

- La predicción Inter se basa en la compensación de movimiento:
  - Se predice un bloque a partir de otro en frames de referencia.



- En H.264, los frames de referencia que pueden usarse para predicción temporal se almacenan en el DPB (Decoded Picture Buffer).
  - Contiene las imágenes decodificadas que se pueden usar como referencia.
- Con los valores del DPB, el encoder forma dos listas para decodificar cada frame :
  - List 0 (L0): Frames anteriores.
  - List 1 (L1): Frames posteriores.
- El orden de decodificación debe ser distinto del orden de visualización para permitir a los B-frames acceder a un frame futuro como referencia.

- Ej: POC (Picture Order Count) es el orden de visualización de los frames.



Pero los frames B tienen una referencia al frame P, y el P al I. Así que a la hora de codificar, se enviaría:



Y ahora en el decoder:

## 1- Decodificar el frame I0.

DPB: { I0 }

## 2- Se contruyen las L0 y L1 para P3:

L0 = [ I0 ]                      (pasado)  
L1 = (vacía)                      (P no usa futuro)

## 3- Se decodifica el frame P3.

DPB: { I0, P3 }

## 4- Se construyen las L0 y L1 para B1:

L0 = [ I0 ]  
L1 = [ P3 ]

5- Se decodifica el frame B1. Como B1 no se usa como referencia en otro frame, no se actualiza el DPB.

DPB: { I0, P3 }

6 - Para B2 lo mismo que para B1.

Al final el orden de visualización será:

I0 → B1 → B2 → P3

mientras que el orden de decodificación fue:

I0 → P3 → B1 → B2

- H.264 permite dimensión variable de los bloques. Puede buscarse similitudes en bloques de 16x16, 16x8, 8x16, 8x8, 4x4, 4x8, 8x4.
- Esto hace que para la predicción de un MB se puedan tener varios vectores de movimiento.
  - Cada uno apuntando a un frame de referencia diferente.
  - En el caso de un MB de tipo B, además puede tener un sub-bloque con dos vectores de movimiento a la vez, uno apuntando a un frame en L0 y otro en L1.
  - La predicción en este caso sería:

$$Pred(x, y) = \frac{Pred_{L0}(x, y) + Pred_{L1}(x, y)}{2}$$

- En H.264 el vector de movimiento tiene una resolución de hasta  $\frac{1}{4}$  de píxel.
  - En estándares antiguos como H.261 o H.263 solo se podía usar valores enteros.
- Interpolación precisa mediante filtros específicos para:
  - Medio píxel
  - Cuarto de píxel

- Al igual que en la predicción Intra, se obtiene un residuo tras la predicción temporal.
  - Residuo = Bloque original – Bloque predicho
- Este residuo es el que se codifica.
- Además hay que enviar el propio vector de movimiento. Pero no se envían tampoco entero. Lo que se envía es:

$$\Delta MV = MV_{\text{real}} - MVP$$

Donde  $MV_{\text{real}}$  es el vector de movimiento real y  $MVP$  es un vector de movimiento predicho usando los vecinos del bloque ya decodificados.

- Normalmente el movimiento será similar y así  $\Delta MV$  será pequeño.

- Para calcular MVP (Motion Vector Prediction) se usan los vecinos del bloque ya decodificados:
  - Izquierda
  - Arriba
  - Arriba-derecha
  - Arriba-izquierda
- Si alguno de ellos no tiene vector de movimiento se usa el de los demás.
- Si ninguno de ellos lo tiene, se establece que  $MVP = (0, 0)$ .

- En H.264 hay otros dos modos de predicción temporal especiales, DIRECT y SKIP.
- DIRECT: No se envía vector de movimiento, sino que este se deriva automáticamente:
  - Heredándolo de un bloque equivalente en un frame futuro y ajustándolo según la diferencia de tiempo (DIRECT TEMPORAL).
    - El mismo bloque aparece en el futuro con un vector de movimiento.
  - o generandose a partir de los bloques vecinos (DIRECT ESPACIAL) si no se puede usar el anterior.
  - Solo se puede aplicar en B-Frames.

- SKIP: Es como el DIRECT, pero tampoco se envía residuo.
- En P-Frames, como no existe DIRECT, significa que el  $MV = 0$ .
  - Equivale a decir que el mismo bloque exacto está en un frame anterior.
- En B-Frames se calcula el valor del  $MV$  como si fuese DIRECT TEMPORAL, y si no se puede se usa DIRECT ESPACIAL.
  - Muy útil para zonas estáticas que casi no se mueven, o se mueven muy poco.

- Tanto si se utilizó una predicción espacial como temporal, al final se obtendrá un residuo (y un vector de movimiento en algunos casos) que hay que codificar.
- Para ello en H.264 se usa una cadena matemática que implica hacer una transformada, una cuantificación y una codificación entrópica.
- En H.264 en lugar de usar la DCT, se usa una aproximación que usa solo enteros:

$$Y = C \cdot R \cdot C^T$$

En este caso la matriz  $C$  es:

$$C = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & -1 & -2 \\ 1 & -1 & -1 & 1 \\ 1 & -2 & 2 & -1 \end{bmatrix}$$

- Esta transformada tiene unas ventajas sobre la DCT:
  - Sin errores acumulativos de redondeo.
  - Buena en bloques pequeños.
- Pero también desventajas:
  - Modela peor las bajas frecuencias.
  - Mayor sensibilidad al ruido.
  - Artefactos de efecto bloque mas visibles.

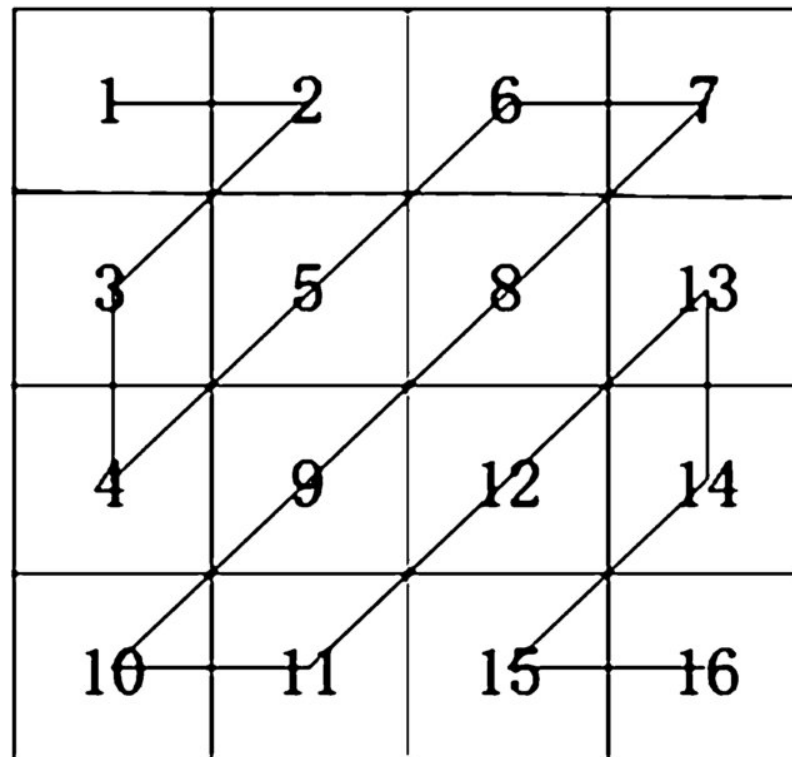
- En H.264 la cuantificación se controla mediante el parámetro  $QP \in [0, 51]$ .

$$\hat{Y}_{i,j} = \text{round} \left( \frac{Y_{i,j}}{Qstep(QP)} \right)$$

donde  $Qstep = 2^{\frac{QP-4}{6}}$

- Ojo, es una relación logarítmica:
  - Cada +6 en QP  $\rightarrow$  duplica el paso de cuantificación.

- Tras cuantificar los valores, los coeficientes se reorganizan siguiendo un zig-zag adaptado a 4x4.
- Esto prepara los datos para la codificación entrópica (como en JPEG).



- Además de los residuos, es necesario codificar los vectores de movimiento.
- Lo que se envía es:
  - $\Delta MV$ : la diferencia con el  $MVP$ .
  - El `ref_idx`: el frame de referencia.

- H.264 incorpora dos métodos diferentes de codificación entrópica.
- CAVLC (Context Adaptive Variable Length Coding)
  - Complejidad baja
  - Usa estadísticas locales (coeficientes no nulos, signo, etc.)
- CABAC (Context Adaptive Binary Arithmetic Coding)
  - Mucho más eficiente (reduce bitrate entre 10–15%)
  - Modelado probabilístico fino
  - Códigos aritméticos adaptativos
  - Requiere mayor potencia de cálculo

- En H.264, un Profile define qué herramientas de codificación están permitidas para un codificador o decodificador.
- Hay muchos profiles diferentes en H.264, pero los principales son:
  - Baseline Profile
  - Main Profile
  - High Profile

### Baseline



El más simple. Sin B-frames, sin CABAC.



Para videoconferencia y móviles antiguos.

### Main



Añade B-frames y CABAC. Buen equilibrio. Para broadcast DVB.



### High



El más común hoy. Añade transformada 8x8 y otras mejoras. Para streaming y Blu-ray. (Ofrece ~10-15% compresión que Main).



## ■ Baseline

- Sin CABAC.
- Sin B-frames.
- Menor complejidad.
- Para videoconferencia, móviles antiguos.

## ■ Main

- Añade soporte para B-frames.
- Usa CABAC.
- Mejor eficiencia.
- Para televisiones, broadcast.

## ■ High

- Permite una transformada 8×8 en vez de 4x4.
- Sobre un 10-15% mejor compresión que el Main.
- Para streaming, Blu-ray, etc.
- Es el perfil más usado en la actualidad.

- Además de los profiles, en H.264 existe otro concepto relacionado aunque diferente: los niveles.
- Un nivel (Level) indica la complejidad máxima que puede manejar un decodificador.
- Se usan para asegurar la compatibilidad entre codificadores y decodificadores.
  - Por ejemplo un móvil de gama baja quizás soporta el perfil High pero solo si el bitrate máximo es de 20 Mbps.
- En el estándar se definen hasta 16 niveles.

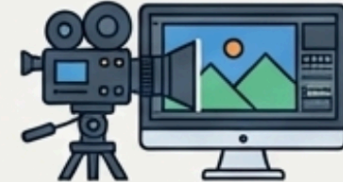
- Cada nivel fija unos máximos para parámetros como:
  - MaxMBPS: Macroblocks por segundo. Límite de  $\text{fps} \times \text{resolución}$ .
  - MaxFS: Macroblocks por frame. Límite de resolución.
  - MaxBR: Bitrate máximo
  - MaxDPB: Tamaño máximo del buffer de decodificación. Límite de cuántos frames de referencia se pueden guardar.
  - MaxVmvR: Rango de los vectores de movimiento. Límite al desplazamiento permitido.

# Codificación Niveles

## Niveles 6.x

**Resolución:** 8K

**Uso:** Masterización profesional, aplicaciones de ultra-alta resolución.



## Niveles 5.x

**Resolución:** 4K (UHD)

**Uso:** Cine digital, streaming premium 4K.



## Niveles 4.x

**Resolución:** 1080p

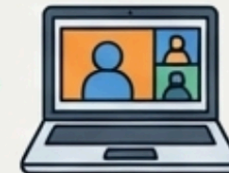
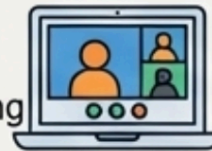
**Uso:** El estándar de facto para HD. Blu-ray (High@4.1), la mayoría de TV y streaming actual.



## Niveles 3.x

**Resolución:** SD, 720p

**Uso:** Webcams modernas, streaming básico, IPTV.



## Niveles 1.x - 2.x

**Resolución:** QCIF, SD (hasta ~480p)

**Uso:** Videollamadas antiguas, cámaras IP básicas, streaming de muy bajo bitrate.



## ■ Niveles 1.0 a 1.3

- Vídeo muy pequeño: QCIF, CIF.
- Usado en videollamadas antiguas, cámaras IP, móviles 2000–2010.
- Bitrate muy bajo.
- DPB pequeño → pocas referencias.

## ■ Niveles 2.0 a 2.2

- SD baja (360p–480p).
- Adecuado para streaming ligero.
- Hasta 4–6 Mbps.
- Más MVs y más DPB permitido.

## ■ Nivel 3.0 – 3.2

- Introduce soporte completo para SD real y 720p.
- Hasta 14–20 Mbps.
- Usado en webcams modernas, streaming básico, TV-IP.

## ■ Nivel 4.0 – 4.2

- 1080p hasta 120 fps.
- Bitrates de hasta 50–62 Mbps.
- Blu-ray = High Profile Level 4.1.
- La mayoría de televisiones aceptan hasta Level 4.1 o 4.2.

### ■ Niveles 5.0 – 5.2

- Destinados a 4K.
- Bitrates muy altos: 100–400 Mbps.
- Usado en cine digital, cámaras, streaming premium.

### ■ Niveles 6.0 – 6.2

- Para 8K.
- Muchísima carga computacional y bitrate (hasta 2.4 Gbps).
- Usado solo en masterización profesional.

- Tras la codificación (predicción, transformada, cuantificación, entropía), los datos deben organizarse para transmitirlos o almacenarlos.
- H.264 usa para ello las NAL Units (Network Abstraction Layer Units).
- Separan claramente:
  - VCL (Video Coding Layer → datos codificados)
  - NAL (Network Layer → empaquetado y transporte)

- Una NALU es la unidad básica del bitstream de H.264.
- Cada NALU contiene:
  - Cabecera (1 byte). Compuesta por:
    - f: forbidden\_zero\_bit. 1 bit. Siempre 0.
    - nal\_ref\_idc: 2 bits. Indica si la NALU es necesaria para una decodificación futura.
    - type: 5 bits. Tipo de NALU.
  - Payload: datos codificados
- Permite que H.264 funcione sobre múltiples contenedores: MP4, MKV, MPEG-TS, RTP/RTSP, etc.

- Tipos más importantes de NALUs:
  - SPS (Sequence Parameter Set): Contiene parámetros globales de la secuencia (perfil, nivel, tamaño).
  - PPS (Picture Parameter Set): Parámetros para las slices de un vídeo.
  - Slices (tipo 1 o 5): Los datos codificados de cada frame.
    - Tipo 5: IDR (punto de acceso aleatorio)
  - SEI: Información adicional (timestamps, HDR, metadata).

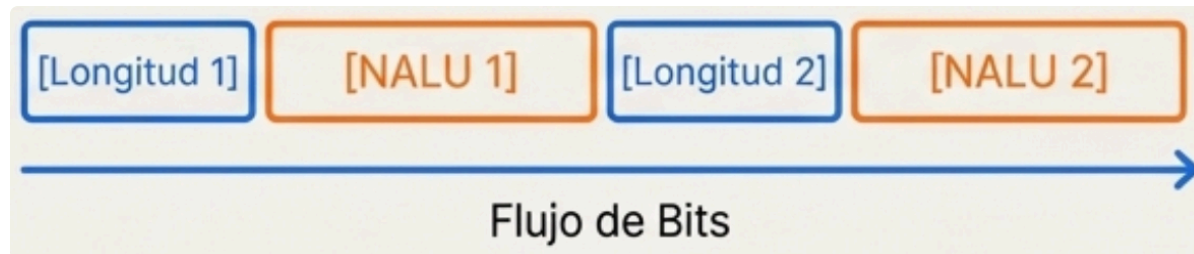
- Un Access Unit agrupa todas las NALUs necesarias para decodificar un frame.
- Normalmente un AU contiene:
  - Una o más NALUs
  - Opcionalmente SEI, etc.
- Cada Access Unit = un frame completo.

- Las NALUs se pueden empaquetar de dos formas diferentes:
  - Annex B (raw .264, TS, RTP)
    - Cada NALU empieza con un start code:
    - Fácil de analizar en streaming.
    - Se usan en RTP/RTSP, DVB, etc.



### ■ MP4 / MOV (ISO BMFF)

- No usan start codes.
- Cada NALU se precede con un campo de longitud que indica el tamaño de la NALU.
- Se usa más para almacenamiento y reproducción.

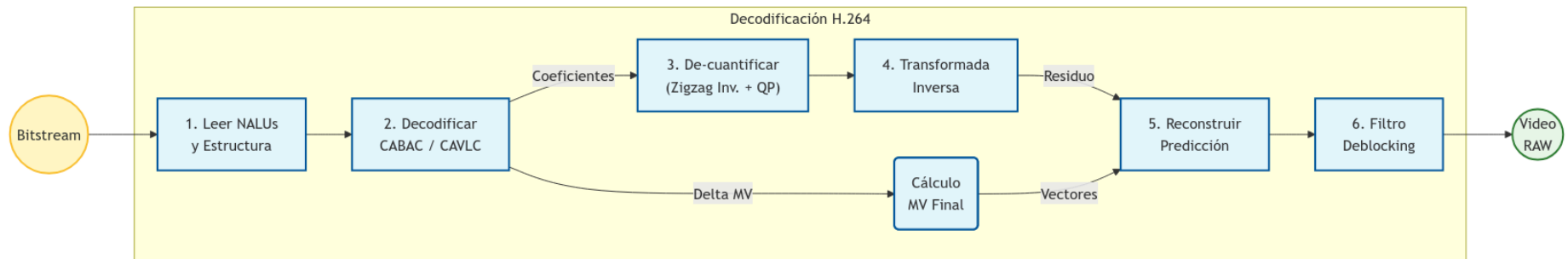


- Se puede cambiar el formato de empaquetado de uno a otro.
- El formato MP4 es más estructurado y tiene metadatos, lo que permite un acceso aleatorio más completo.
- Hoy en día en streaming moderno se usa igualmente el formato MP4, ya que es una capa superior la que se encarga de ello (fMP4, fragmented MP4).

# Decodificación

## Pasos

- Tras recibir el bitstream y extraer las NAL Units, el decodificador H.264 realiza esencialmente el proceso inverso al codificador:



### 1. Leer NALUs

- Se reconstruye la estructura de slices y MBs.

### 2. Decodificar CABAC / CAVLC

- Se obtienen los coeficientes cuantificados y los  $\Delta MV$ .

### 3. De-cuantificar

- Se hace el zigzag inverso y se de-cuantifica usando el mismo QP que en el codificador.

#### 4. Transformada inversa

- Se hace la transformada inversa para obtener los coeficientes originales.

#### 5. Reconstruir predicción

- Se reconstruye la predicción usando los  $MV$  y se suma el residuo.

#### 6. Aplicar filtro deblocking

- Se aplica el filtro deblocking para suavizar bordes y reducir discontinuidades.
- Cada frame decodificado se envía para su visualización cuando corresponda y se guarda en el DPB si es referencia de algún otro.

- Filtro aplicado después de reconstruir cada MB.
- Reduce discontinuidades entre bloques y suaviza bordes:
  - Efecto “blocking” típico de la DCT
- Mejora notablemente la calidad visual.
- Es obligatorio en H.264.

